



React'in Felsefesi, Temel Kavramları ve Araştırma Başlıkları

Modern web geliştirme dünyasında kullanıcı arayüzlerini hızlı, yeniden kullanılabilir ve yönetilebilir şekilde geliştirebilmek büyük önem taşır. Facebook'un 2013 yılında tanıttığı React, bu ihtiyaca getirdiği yenilikçi yaklaşımla günümüz front-end ekosisteminin temel taşlarından biri hâline gelmiştir.

Bu hafta, React yazmaya başlamadan önce React'in mantığını, neden ortaya çıktığını, hangi problemleri çözdüğünü, bileşen mimarisini ve ekosistem yapısını kavramsal düzeyde anlamayı hedefliyoruz.

Komutlara ve kod yazmaya geçmeden önce React'in ne olduğunu bilmek, ileride oluşturacağımız projelere sağlam bir temel hazırlayacaktır.

Bu doküman, React'ın felsefesini, mimarisini, kullanım amacını ve araştırma yapılması gereken temel başlıkları açıklamak için hazırlanmıştır.

React Nedir?

React; web uygulamalarında kullanıcı arayüzleri (UI) oluşturmak için kullanılan, **bileşen tabanlı, deklaratif** bir JavaScript kütüphanesidir.

Nerelerde Kullanılır?

- ✓ Web uygulamaları (SPA – Single Page Application)
- ✓ Mobil uygulamalar (React Native)
- ✓ Desktop uygulamalar (Electron + React)
- ✓ Dashboard'lar, Admin panelleri
- ✓ Gerçek zamanlı arayüzler
- ✓ Tasarım sistemleri ve component library'ler

React'in Yaygın Kullanım Sebepleri

- ⚡ **Hızlı render** (Virtual DOM sayesinde)
- 🧩 **Component mantığıyla yeniden kullanılabilirlik**
- 🔄 **Tek yönlü veri akışıyla tahmin edilebilir yapı**
- 🌍 **Büyük ekosistem ve topluluk desteği**
- 🎨 **UI odaklı geliştirme modeli**
- 🛠️ **DevTools ve modern çalışma standartlarına uyum**

React Neyi Çözer? Felsefesi Nedir?

Modern web uygulamaları karmaşıktıkça:

- Çok sayıda UI parçası oluştu
- Yönetilmesi zor state (durum) yapıları doğdu

- Kod tekrarı arttı
- Güncellemeler maliyetli hâle geldi

React bu problemlere şu prensiplerle çözüm üretir:

★ React Felsefesi

1. **UI = Component'ler** → Her şey küçük parçalara ayrılır.
2. **Deklaratif yaklaşım** → "Nasıl?" değil "Ne?"ye odaklanırsın.
3. **Tek yönlü veri akışı** → Yönetmesi kolay sistem.
4. **Virtual DOM** → DOM güncellemesi minimum maliyetle yapılır.
5. **Stateless UI + State yönetimi** → Daha tutarlı arayüzler.

3 React'ın Temel Kavramları (Araştırma Başlıkları)

Aşağıdaki başlıklar ekip çalışmanızda ödev olarak verilebilir.
Her başlık, "Nedir?" + "Neden önemlidir?" şeklinde araştırılmalıdır.

◆ 3.1. Component (Bileşen) Mantığı

Araştırma soruları:

- Component nedir?
 - Neden bütün UI component olarak düşünülür?
 - Fonksiyonel ve Class component farkı?
 - Component reusability (yeniden kullanılabilirlik) neden önemlidir?
-

◆ 3.2. Props – Veri Aktarımı

Araştırma soruları:

- Props nedir?
 - Component'lar arası neden props aktarıyoruz?
 - Props değiştirilebilir mi? (Hayır → immutable)
 - Props drilling problemi nedir?
-

◆ 3.3. State – Uygulama Durumu

Araştırma soruları:

- State nedir, ne zaman kullanılır?
 - Neden direkt değiştirilemez?
 - State ile props farkı ne?
 - Çok fazla state kullanılınca ne sorunlar olur?
-

◆ 3.4. Virtual DOM

Araştırma soruları:

- Virtual DOM nedir?
 - React neden doğrudan DOM ile çalışmaz?
 - Reconciliation ne demektir?
 - Virtual DOM performansı nasıl artırır?
-

◆ 3.5. Hooks – Modern React'in Kalbi

Araştırma soruları:

- Hooks neden çıktı?
 - useState, useEffect, useRef, useMemo ne işe yarar?
 - Neden class component yerine hook'lar tercih ediliyor?
-

◆ **3.6. JSX – React'in Sihri**

Araştırma soruları:

- JSX nedir? HTML ile farkı nedir?
 - Neden JS içinde HTML benzeri yapı kullanılır?
 - JSX olmasa React nasıl olurdu? (createElement yapısı)
-

◆ **3.7. React Ekosistemi (Router, State, Styling)**

Araştırma soruları:

- React Router nedir?
 - Context API nedir?
 - Küçük/Büyük projelerde state yönetimi nasıl değişir?
 - UI/Component kütüphaneleri (MUI, Tailwind, Chakra, Ant Design)
-

◆ **3.8. Build Süreçleri – Create React App vs Vite vs Next.js**

Araştırma soruları:

- React kendi başına bir framework mü? (Hayır, sadece UI library)

- CRA neden artık pek önerilmiyor?
 - Vite'nin avantajı nedir?
 - Next.js neden popüler?
-

4 React Öğrenirken Önemli Olan Mantıksal Yapılar

Bu bölüm araştırma değil, "neden önemli?" kısmı:

✨ Component Thinking

UI'ı küçük, bağımsız parçalar hâline getirmek.
Önemli çünkü: okunabilirlik + sürdürülebilirlik sağlar.

✨ State Yönetimi

Uygulamanın durumunu kontrol altında tutmak.
Önemli çünkü: karmaşık uygulamalarda veri kaosu yaşanmaz.

✨ Side Effect Yönetimi

API çağrıları, zamanlayıcılar gibi dış dünya etkileşimleri.
Önemli çünkü: UI'nin doğru zamanda güncellenmesini sağlar.

✨ Performans Optimizasyonu

Gereksiz render engellenir.
Önemli çünkü: büyük uygulamalarda hız farkı devasa olur.

5 React Öğrenirken Kullanabileceğin Kaynaklar

(Resmî ve güvenilir kaynaklar)

Resmi Kaynaklar

-  React Resmi Dokümantasyon
<https://react.dev>
(Yeni doküman → modern, anlaşılır tasarım)
 -  Eski React Dokümantasyonu (bazı konular hâlâ burada)
<https://legacy.reactjs.org>
-

Uygulamalı Öğrenme Siteleri

- Frontend Mentor
 - CodeSandbox
 - Stackblitz
 - Scrimba
-

Ekosistem Kaynakları

- React Router Docs → <https://reactrouter.com>
- Next.js Docs → <https://nextjs.org/docs>
- Redux Toolkit → <https://redux-toolkit.js.org>